

LeetCode 每日背诵

每题只保留：极简题目、思路、关键数据结构、注意点、核心代码。代码尽量按当前源码保留，方便直接背和改。

No.1 哈希

两数之和

题目：给数组和 target，返回两个数之和等于 target 的下标。

- 文件：No.1_两数之和.cpp
- 源码注释解析：
 1. 这题的关键是利用 `unordered_map` 查找平均 $O(1)$ 的特点，把暴力双循环降成一遍扫描。
 2. 哈希表的 key 存数组里的值，value 存这个值对应的下标。
 3. 遍历到 `nums[i]` 时，不是先存当前数，而是先查 `target - nums[i]` 是否已经出现。
 4. 如果补数已经在哈希表里，说明当前数和之前某个数可以组成答案，直接返回两个下标。
 5. 如果没有找到补数，再把当前数和下标存进去，给后面的元素使用。
- `unordered_map` 复习：
 1. `mp[key] = value` 可以插入或修改。
 2. `mp.find(key)` 查找 key，找不到时返回 `mp.end()`。
 3. `mp.count(key)` 可以判断 key 是否存在。
 4. 遍历哈希表可以用 `for (auto& item : mp)`，其中 `item.first` 是 key，`item.second` 是 value。
 5. 删除元素可以用 `mp.erase(key)`。
- 易错点：先查再存，避免同一个元素被用两次。
- 复杂度：时间 $O(n)$ ，空间 $O(n)$ 。

```
#include<iostream>
#include<vector>
#include<unordered_map>

using namespace std;

vector<int> twosum(vector<int> &nums, int target){
    unordered_map<int , int> unmap;
    for(int i = 0; i < nums.size(); i ++){
        auto it = unmap.find(target - nums[i]);
        if(it != unmap.end()){
            return {i, it->second};
        }
        unmap[nums[i]] = i;
    }
    return {-1, -1};
}
```

字母异位词分组

题目：把互为字母异位词的字符串分到同一组。

- 文件：No.1_字母异位词分组.cpp
- 源码注释解析：
 1. 虽然源码里没有长注释，但代码思路很固定：字母异位词只是字母顺序不同，排序后一定相同。
 2. 对每个字符串复制一份，排序后作为哈希表 key。
 3. 哈希表 value 是一个 `vector<string>`，用来保存所有属于同一组的原始字符串。
 4. 最后遍历哈希表，把每个 value 放进答案。
- 易错点：排序的是复制出来的 key，答案里要放原字符串。
- 复杂度：设字符串平均长度为 k，时间 $O(n * k \log k)$ ，空间 $O(nk)$ 。

```
#include<iostream>
#include<string>
#include<vector>
#include<unordered_map>
#include<algorithm>

using namespace std;

vector<vector<string>> groupAnagrams(vector<string> &nums){
    unordered_map <string, vector<string>> unmap;
    for(int i = 0; i < nums.size(); i++){
        string s = nums[i];
        sort(s.begin(), s.end());
        unmap[s].push_back(nums[i]);
    }
    vector<vector<string>> result;
    for(auto it = unmap.begin(); it != unmap.end(); it++){
        result.push_back(it->second);
    }
    return result;
}
```

最长连续序列

题目：返回数组中最长连续数字序列的长度。

- 源码注释解析：
 1. 把所有数字放进 `unordered_set`，这样判断某个数字是否存在就是平均 $O(1)$ 。
 2. 如果一个数 x 的前一个数 $x - 1$ 存在，说明 x 不是连续序列的起点，直接跳过。
 3. 只有当 $x - 1$ 不存在时，才从 x 开始向后查 $x + 1, x + 2 \dots$ 。
 4. 每找到一个连续数字，当前长度加一。
 5. 每个连续段只会从最小值开始统计一次，所以整体仍然是 $O(n)$ 。
- 易错点：每个连续段起点的长度要重新从 1 开始。
- 复杂度：时间 $O(n)$ ，空间 $O(n)$ 。

```
#include<iostream>
#include<vector>
#include<unordered_set>

using namespace std;

int findlong(vector<int> &nums){
    if(nums.size() == 0){
        return 0;
    }
    unordered_set<int> unset;
    for(int i = 0; i < nums.size(); i++){
        unset.insert(nums[i]);
    }
    int result = 0, temp = 1, i;
    for(auto it = unset.begin(); it != unset.end(); it++){
        if(unset.find(*it - 1) != unset.end()) continue;
        i = 1;
        while(unset.find(*it + i) != unset.end()){
            i++;
            temp++;
        }
        result = max(result, temp);
    }
    return result;
}
```

No.2 双指针

三数之和

题目：找出数组中所有和为 0 且不重复的三元组。

- 源码注释解析：
 1. 先排序，让数组有序，后面才能使用双指针。
 2. 枚举第一个数 `nums[i]`，剩下两个数在区间 `[i + 1, n - 1]` 里找。
 3. 左指针 `l` 指向较小数，右指针 `r` 指向较大数。
 4. 如果三数之和等于 0，记录答案，并跳过左右两边的重复值。
 5. 如果和大于 0，说明右边太大，`r--`。
 6. 如果和小于 0，说明左边太小，`l++`。
 7. 固定的第一个数也要去重，否则答案会重复。
- 易错点：固定数去重要写 `continue`，不能在 `if` 后面多写分号。
- 复杂度：时间 $O(n^2)$ ，空间 $O(1)$ 。

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
```

```
vector<vector<int>> ThreeSum(vector<int> & nums){
    vector<vector<int>> ans;
    sort(nums.begin(), nums.end());

    for(int i = 0; i < nums.size(); i++){
        if(i > 0 && nums[i] == nums[i - 1]);
        int l = i + 1, r = nums.size() - 1;
        while(l < r){
            int sum = nums[i] + nums[l] + nums[r];
            if(sum == 0){
                ans.push_back({nums[i], nums[l], nums[r]});
                while(l < r && nums[l] == nums[l + 1]) l++;
                while(l < r && nums[r] == nums[r - 1]) r--;
                l++;
                r--;
            }else if(sum > 0) r--;
            else l++;
        }
    }
    return ans;
}
```

接雨水

题目：给柱子高度，计算最多能接多少雨水。

- 源码注释解析：
 1. 对某个位置来说，它能接的水由左右两边最高柱子的较小值决定。
 2. 用左指针 `l` 和右指针 `r` 从两端向中间走。
 3. `lm` 表示从左到当前位置遇到的最高柱子，`rm` 表示从右到当前位置遇到的最高柱子。
 4. 如果 `lm < rm`，说明左边短板已经确定，左侧位置最多只能接到 `lm`，所以结算左边。
 5. 否则右边短板已经确定，结算右边。
 6. 每次累加当前短板高度减去当前位置高度。
- 易错点：先更新 `lm / rm`，再累加当前位置水量。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
#include<iostream>
#include<vector>

using namespace std;

int JieShui(vector<int> & height){
    int l = 0, r = height.size() - 1;
    int lm = 0, rm = 0, ans = 0;
    while(l < r){
        lm = max(lm, height[l]);
        rm = max(rm, height[r]);
        if(lm < rm){
            if(height[l] < lm) ans += lm - height[l];
        }
        l++;
    }
    while(r > l){
        rm = max(rm, height[r]);
        lm = max(lm, height[l]);
        if(lm < rm){
            if(height[r] < rm) ans += rm - height[r];
        }
        r--;
    }
    return ans;
}
```

```

        l++;
    }else{
        if(height[r] < rm) ans += rm - height[r];
        r--;
    }
}
return ans;
}

```

盛最多水的容器

题目：找两条线，使容器装水面积最大。

- 源码注释解析：
 1. 容器面积等于 $\min(\text{height}[l], \text{height}[r]) * (r - l)$ 。
 2. 宽度会随着指针移动不断变小，所以想让面积变大，只能尝试提高短板。
 3. 因此每次移动较短的一边。
 4. 如果移动较高的一边，短板没有变高，宽度又变小，面积不可能更优。
- 易错点：不要移动高的一边，因为宽度变小且短板没变。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```

#include<iostream>
#include<vector>

using namespace std;

int Maxarea(vector<int> &height){
    int l = 0, r = height.size() - 1;
    int lm = 0, rm = 0, ans = 0, temp = 0;
    ans = min(height[l], height[r]) * (r - l);
    while(l < r){
        lm = max(lm, height[l]);
        rm = max(rm, height[r]);
        if(lm < rm){
            while(l < r && height[l] <= lm) l++;
        }else{
            while(l < r && height[r] <= rm) r--;
        }
        temp = min(height[l], height[r]) * (r - l);
        ans = max(ans, temp);
    }
    return ans;
}

```

移动零

题目：把所有 0 移到数组末尾，保持非零顺序。

- 源码注释解析：

1. 用慢指针 `slow` 表示下一个非零元素应该放的位置。
 2. 用快指针 `fast` 从左到右扫描所有元素。
 3. 当 `nums[fast] != 0` 时，把它交换到 `slow` 位置，然后 `slow++`。
 4. 这样所有非零元素会保持原来的相对顺序，所有 0 会自然被换到后面。
- 易错点：遇到非零才交换并移动慢指针。
 - 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
#include<iostream>
#include<vector>

using namespace std;

void MoveZero(vector<int> &nums){
    int l = 0, r;
    int n = nums.size();
    while(l < n && nums[l] != 0) l++;
    r = l + 1;
    while(r < n){
        while(r < n - 1 && nums[r] == 0) r++;
        swap(nums[l], nums[r]);
        l++;
        r++;
    }
}
```

No.3 滑动窗口

找到字符串中所有字母异位词

题目：找出 `s` 中所有 `p` 的异位词起始下标。

- 源码注释解析：
 1. 使用固定长度滑动窗口，窗口大小等于 `p.size()`。
 2. 窗口在 `s` 上从左往右滑动，每次判断窗口内字符计数是否和 `p` 一致。
 3. 因为题目只包含小写英文字母，所以可以开两个长度为 26 的数组。
 4. `pcount[26]` 统计 `p` 中每个字母出现次数，这个数组固定不变。
 5. `scount[26]` 统计当前窗口内 `s` 的字母出现次数，窗口滑动时动态更新。
 6. 如果 `scount == pcount`，说明当前窗口是 `p` 的异位词，记录窗口左边界。
 7. `vector<int>` 可以直接用 `==` 比较 26 个字母的计数是否完全相等。
 8. 边界要注意最后一个窗口，循环结束后仍然要判断一次，或者把循环写成能覆盖最后窗口的形式。
- 易错点：最后一个窗口也要判断；`p.size() > s.size()` 直接返回空。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
#include<iostream>
#include<string>
```

```

#include<vector>

using namespace std;

vector<int> findAnagrams(string &s, string &p){
    vector<int> ans;
    vector<int> scount(26, 0);
    vector<int> pcount(26, 0);
    int l = 0, r = p.size(), n = s.size();
    if(r > n){
        return ans;
    }
    for(int i = 0; i < r; i++){
        scount[s[i] - 'a']++;
        pcount[p[i] - 'a'] ++;
    }
    while(r < n){
        if(pcount == scount){
            ans.push_back(l);
        }
        scount[s[l] - 'a']--;
        scount[s[r] - 'a']++;
        r++;
        l++;
    }
    if(pcount == scount){
        ans.push_back(l);
    }
    return ans;
}

```

无重复字符的最长子串

题目：返回没有重复字符的最长子串长度。

- 源码注释解析：
 1. 窗口定义可以理解为 $[l, r]$ ，窗口内永远保存不含重复字符的连续子串。
 2. 用数组 `cnt[300]` 当作 ASCII 字符哈希表，记录窗口内每个字符出现次数。
 3. 右指针不断扩张窗口，把新字符加入窗口。
 4. 如果新加入的字符导致重复，就不断移动左指针，直到重复字符被移出，窗口重新合法。
 5. 整体逻辑一句话：右指针不断扩张窗口，遇到重复就收缩左指针，全程记录合法窗口最大长度。
- 易错点：更新答案时用当前合法窗口长度 $r - l + 1$ 。
- 复杂度：时间 $O(n)$ ，空间 $O(\text{字符集大小})$ 。

```

#include<iostream>
#include<string>

using namespace std;

int ZiChuan(string &s){

```

```

int l = 0, r = 0, ans = 0;
int unmap[300] = {0};
int n = s.size();
while(r < n){
    while(r < n && unmap[s[r]] < 1){
        unmap[s[r]]++;
        r++;
    }
    ans = max(ans, r - l);
    unmap[s[l]]--;
    l++;
}
return ans;
}

```

No.4 子串

和为 K 的子数组

题目：统计数组中和为 k 的连续子数组个数。

- 源码注释解析：
 1. 题目数组里可能包含正数、负数和 0，所以不能使用普通的伸缩滑动窗口。
 2. 暴力双循环会超时，因此采用“前缀和 + 哈希表”的 $O(n)$ 解法。
 3. 定义 $pre[i]$ 为前 i 项和，并人为规定 $pre[0] = 0$ 。
 4. 子数组 $[l, r]$ 的和为 $pre[r + 1] - pre[l]$ 。
 5. 如果子数组和要等于 k ，就有 $pre[r + 1] - pre[l] = k$ 。
 6. 变形得到 $pre[l] = pre[r + 1] - k$ 。
 7. 遍历到当前位置时，当前前缀和记为 pre ，只需要查之前出现过多少次 $pre - k$ 。
 8. 哈希表记录的是“某个前缀和出现过几次”，出现几次就能新增几个符合条件的子数组。
 9. 初始化 $mp[0] = 1$ ，因为当前缀和刚好等于 k 时，从数组开头到当前位置也算一个答案。
- 易错点：必须初始化 $mp[0] = 1$ ，并且要先查答案，再记录当前前缀和。
- 复杂度：时间 $O(n)$ ，空间 $O(n)$ 。

```

#include<iostream>
#include<vector>
#include<unordered_map>

using namespace std;

int subarraySum(vector<int>& nums, int &k){
    unordered_map<int, int> unmap;
    int ans = 0, pro = 0;
    unmap[0] = 1;
    for(int i = 0; i < nums.size(); i++){
        pro += nums[i];
        auto it = unmap.find(pro - k);
        if(it != unmap.end()){
            ans += it->second;
        }
    }
}

```

```
        unmap[pro]++;
    }
    return ans;
}
```

最小覆盖子串

题目：找 s 中覆盖 t 所有字符的最短子串。

- 源码注释解析：
 1. 先预处理 t ，用 $need$ 数组记录每个字符的需求数量。
 2. $kind$ 记录 t 中不同字符的种类数。
 3. win 记录当前窗口中每个字符的数量。
 4. $match$ 记录当前窗口中已经“数量满足要求”的字符种类数。
 5. 右指针 r 扩张窗口，每加入一个字符，就更新 win 。
 6. 如果某个字符数量刚好达到需求，即 $win[c] == need[c]$ ，说明这个字符种类匹配齐了， $match++$ 。
 7. 当 $match == kind$ 时，说明窗口已经合法，当前窗口覆盖了 t 。
 8. 窗口合法后开始收缩左边界 l ，每次收缩前更新最短子串答案。
 9. 如果移出的左侧字符原本是刚好满足需求的字符，移出后会破坏合法性，所以 $match--$ 。
 10. 最后如果没有找到合法窗口，返回空字符串；否则返回最短窗口。
- 易错点：只有字符数量刚好满足需求时， $match$ 才变化；移出字符前要判断是否会破坏匹配。
- 复杂度：时间 $O(n)$ ，空间 $O(\text{字符集大小})$ 。

```
#include<iostream>
#include<string>

using namespace std;

string minWindow(string &s, string &t){
    int win[128] = {0}, need[128] = {0};
    int kind = 0; // t中不同字符种类
    for(int i = 0; i < t.size(); i++){
        if(need[t[i]] == 0){
            kind++;
        }
        need[t[i]]++;
    }
    int match = 0, start = 0, len = 99999;
    int l = 0, r = 0, n = s.size();
    while(r < n){
        win[s[r]]++;

        // 当前字符数量刚好匹配，匹配种类+1
        if(win[s[r]] == need[s[r]]){
            match++;
        }
        // 窗口合法，收缩左边界
        while(match == kind){
```

```

        // 更新最小窗口
        int temp = r - l + 1;
        if(len > temp){
            len = temp;
            start = l;
        }
        // 如果这个字符是刚好匹配的，移除后匹配数减少
        if(win[s[l]] == need[s[l]]) match--;
        win[s[l]]--;
        l++;
    }
    r++;
}
return len == 99999? "" : s.substr(start, len);
}

```

滑动窗口最大值

题目：返回每个长度为 k 的窗口最大值。

- 源码注释解析：
 1. 优化方案是使用单调递减双端队列 `deque`，整体时间复杂度 $O(n)$ 。
 2. 队列里只存数组下标，不直接存值。
 3. 队列维护规则：队列中下标对应的数值，从队头到队尾严格单调递减。
 4. 因此队头永远是当前窗口最大值的下标。
 5. 新元素入队前，把队尾所有小于等于当前值的下标全部弹出。
 6. 这些被弹出的元素不可能成为后续窗口最大值，因为它们更小，并且位置更靠左。
 7. 窗口右移后，要检查队头下标是否滑出窗口左边界，超出则弹出队头。
 8. 当窗口长度达到 `k` 后，每一步都把 `nums[deque.front()]` 加入答案。
- 易错点：队列里存下标；判断过期用 `que.front() <= i - k`。
- 复杂度：时间 $O(n)$ ，空间 $O(k)$ 。

```

#include<iostream>
#include<vector>
#include<deque>

using namespace std;

vector<int> maxSlidingWindow(vector<int> &nums, int &k){
    deque<int> que;
    vector<int> ans;
    for(int i = 0; i < k; i++){
        while(!que.empty() && nums[i] >= nums[que.back()]){
            que.pop_back();
        }
        que.push_back(i);
    }
    ans.push_back(nums[que.front()]);
    for(int i = k; i < nums.size(); i++){
        while(!que.empty() && nums[i] >= nums[que.back()]){

```

```

        que.pop_back();
    }
    que.push_back(i);
    if(que.front() <= i - k){
        que.pop_front();
    }
    ans.push_back(nums[que.front()]);
}
return ans;
}

```

No.5 普通数组

合并区间

题目：合并所有重叠区间。

- 源码注释解析：
 1. 第一步：先按照每个区间的左端点从小到大排序。
 2. 第二步：逐个遍历区间并合并。
 3. 使用一个新数组保存合并后的结果。
 4. 如果结果数组为空，直接把当前区间放进去。
 5. 否则拿结果数组最后一个区间的右端点，和当前区间左端点比较。
 6. 如果结果最后区间的右端点小于当前左端点，说明没有重叠，直接新增当前区间。
 7. 否则说明两个区间重叠，需要合并。
 8. 合并方式是更新结果最后一个区间的右端点，取两个右端点里的最大值。
- 易错点：比较的是 `ans.back()[1]` 和当前左端点。
- 复杂度：时间 $O(n \log n)$ ，空间 $O(n)$ 。

```

#include<iostream>
#include<vector>
#include<algorithm>

using namespace std;

bool panduan(vector<int> a, vector<int> b){
    return a[0] < b[0];
}

vector<vector<int>> merge(vector<vector<int>> & nums){
    if(nums.size() <= 1){
        return nums;
    }
    sort(nums.begin(), nums.end(), panduan);
    vector<vector<int>> merge;
    for(int i = 0; i < nums.size(); i++){
        int l = nums[i][0], r = nums[i][1];
        if(merge.empty() || merge.back()[1] < l){
            merge.push_back({l, r});
        }else{

```

```

        merge.back()[1] = max(r, merge.back()[1]);
    }
}
return merge;
}

```

最大子数组和

题目：返回连续子数组的最大和。

- 源码注释解析：
 1. 用两个变量。
 2. `cur` 表示“以当前数字结尾的最大连续和”。
 3. `ans` 表示全程记录的最大结果。
 4. 遍历每个数字时，如果前面的和加上当前数，不如只取当前数，说明前面的部分拖后腿，要丢掉前面，从当前数重新开始。
 5. 状态转移就是 `cur = max(cur + nums[i], nums[i])`。
 6. 每次更新完 `cur`，都用它更新全局最大值 `ans`。
- 易错点：初始化用 `nums[0]`，不要用固定哨兵值。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```

#include<iostream>
#include<vector>

using namespace std;

int maxSubArray(vector<int> &nums){
    int pro = -999999, ans = -999999;
    for(int i = 0; i < nums.size(); i++){
        pro = max(pro + nums[i], nums[i]);
        ans = max(ans, pro);
    }
    return ans;
}

```

缺失的第一个正数

题目：找数组中缺失的最小正整数。

- 源码注释解析：
 1. 根据抽屉原理，长度为 n 的数组中，缺失的第一个正整数只可能在 $[1, n + 1]$ 范围内。
 2. 小于等于 0 的数，以及大于 n 的数，都不影响答案。
 3. 使用原地哈希：把范围 $[1, n]$ 内的数字 x 放到它应该在的位置，也就是下标 $x - 1$ 。
 4. 例如数字 1 应该放在下标 0，数字 2 应该放在下标 1。
 5. 遍历数组时，如果当前数在 $[1, n]$ 内，就尝试把它交换到正确位置。
 6. 如果目标位置已经是同一个数字，说明遇到了重复数字，要停止交换，避免死循环。
 7. 归位结束后再遍历数组，找到第一个满足 `nums[i] != i + 1` 的位置，返回 `i + 1`。
 8. 如果所有位置都匹配，说明 $[1, n]$ 都存在，答案就是 $n + 1$ 。

9. 边界重点：这里要用 `while`，不能只用 `if`，因为换过来的新数字可能还需要继续检查并归位。

- 易错点：交换必须用 `while`；遇到重复数字要停，防止死循环。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
#include<iostream>
#include<vector>
#include<algorithm>

using namespace std;

int firstMissingPositive(vector<int>& nums){
    int n = nums.size();
    for(int i = 0; i < n; i++){
        while(nums[i] >= 1 && nums[i] <= n){
            int to = nums[i] - 1;
            if(nums[to] == nums[i]) break;
            swap(nums[to], nums[i]);
        }
    }
    for(int i = 0; i < n; i++){
        if(nums[i] != i + 1) return i + 1;
    }
    return n + 1;
}
```

轮转数组

题目：把数组向右轮转 k 步。

- 源码注释解析：
 1. 使用三步反转。
 2. 第一步：整体反转数组。
 3. 第二步：反转前 k 个元素。
 4. 第三步：反转剩下的后半部分。
 5. 右轮转超过数组长度时会重复，所以要先做 $k \% \text{nums.size}()$ ，消除多余轮转次数。
- 易错点：如果数组可能为空，要先判断 $n == 0$ ，否则 $k \% n$ 会出错。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
#include<iostream>
#include<vector>
#include<algorithm>

using namespace std;

void rotate(vector<int> & nums, int &k){
    reverse(nums.begin(), nums.end());
    reverse(nums.begin(), nums.begin() + k % nums.size());
}
```

```
reverse(nums.begin() + k % nums.size(), nums.end());
}
```

除自身以外数组的乘积

题目：返回除自己外其他元素乘积组成的数组。

- 源码注释解析：
 1. 题目要求不能使用除法，所以要把乘积拆开。
 2. 除 `nums[i]` 以外的乘积，等于 `i` 左边所有数的乘积乘以 `i` 右边所有数的乘积。
 3. 可以用前缀乘积数组记录从开头到当前位置之前的乘积。
 4. 也可以用后缀乘积数组记录从末尾到当前位置之后的乘积。
 5. 最终 `ans[i] = 左侧全部乘积 * 右侧全部乘积`。
 6. 更适合背诵的写法是只用答案数组：第一遍从左到右把左侧乘积写进 `ans[i]`，第二遍从右到左用变量 `right` 乘上右侧乘积。
 7. 这样不需要额外开前缀数组和后缀数组，空间更省。
- 易错点：推荐背“一遍左乘积，一遍右乘积”，不用除法。
- 复杂度：时间 $O(n)$ ，额外空间 $O(1)$ ，不算答案数组。

```
#include<iostream>
#include <vector>

using namespace std;

vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> cpro(n, 0), clro(n, 0), ans(n, 0);
    cpro[0] = nums[0];
    for(int i = 1; i < n - 1; i++){
        cpro[i] = cpro[i - 1] * nums[i];
    }
    clro[0] = nums[n - 1];
    for(int i = 1; i < n - 1; i++){
        clro[i] = clro[i - 1] * nums[n - i - 1];
    }
    ans[0] = clro[n - 2];
    for(int i = 1; i < n - 1; i++){
        ans[i] = cpro[i - 1] * clro[n - i - 2];
    }
    ans[n - 1] = cpro[n - 2];
    return ans;
}
```

No.6 矩阵

矩阵置零

题目：某元素为 0，则所在行列全部置 0。

```
/*
```

思路：先记录哪些行列需要置零，再统一修改矩阵，避免边遍历边改造成连锁影响。

一共需要5步：1. 扫描第一列，检查是否有 0

2. 扫描第一行，检查是否有 0

3. 使用第一行和第一列作为“标记位”，扫描剩下的 $[1\dots m-1][1\dots n-1]$

4. 根据第一行/列的标记位，执行置零

5. 最后单独处理第一行和第一列（因为它们被用来存标记了）

关键数据结构：可以用 row/col 标记数组，进阶可用矩阵第一行第一列原地充当标记。

注意：第一行和第一列本身是否有 0 要提前单独记录。

```
*/
```

```
#include "MatrixTestUtils.hpp"
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
    void setZeroes(vector<vector<int>>& matrix) {
```

```
        int m = matrix.size();
```

```
        int n = matrix[0].size();
```

```
        bool fm = 0, fn = 0;
```

```
        for(int i = 0; i < m; i++){
```

```
            if(matrix[i][0] == 0) fm = 1;
```

```
        }
```

```
        for(int i = 0; i < n; i++){
```

```
            if(matrix[0][i] == 0) fn = 1;
```

```
        }
```

```
        for(int i = 1; i < m; i++){
```

```
            for(int j = 1; j < n; j++){
```

```
                if(matrix[i][j] == 0){
```

```
                    matrix[i][0] = 0;
```

```
                    matrix[0][j] = 0;
```

```
                }
```

```
            }
```

```
        }
```

```
        for(int i = 1; i < m; i++){
```

```
            for(int j = 1; j < n; j++){
```

```
                if(matrix[i][0] == 0 || matrix[0][j] == 0){
```

```
                    matrix[i][j] = 0;
```

```
                }
```

```
            }
```

```
        }
```

```
        if(fm == 1){
```

```
            for(int i = 0; i < m; i++){
```

```
                matrix[i][0] = 0;
```

```
            }
```

```
        }
```

```
        if(fn == 1){
```

```
            for(int i = 0; i < n; i++){
```

```
                matrix[0][i] = 0;
```

```
            }
```

```
        }
```

```

    }
}
};

```

螺旋矩阵

题目：按顺时针螺旋顺序返回矩阵元素。

```

/*
思路：维护上下左右四个边界，按右、下、左、上的顺序一圈圈向内遍历。
关键数据结构：top/bottom/left/right 四个边界变量，用来限定当前还没遍历的矩阵范围。
注意：每走完一个方向就收缩对应边界，并及时判断是否越界。
*/

#include "MatrixTestUtils.hpp"

using namespace std;

class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        int top = 0, bottom = matrix.size() - 1;
        int left = 0, right = matrix[0].size() - 1;
        vector<int> ans;

        while(top <= bottom){
            for(int i = left; i <= right; i++){
                ans.push_back(matrix[top][i]);
            }
            if(++top > bottom) break;

            for(int i = top; i <= bottom; i++){
                ans.push_back(matrix[i][right]);
            }
            if(--right < left) break;

            for(int i = right; i >= left; i--){
                ans.push_back(matrix[bottom][i]);
            }
            if(--bottom < top) break;

            for(int i = bottom; i >= top; i--){
                ans.push_back(matrix[i][left]);
            }
            if(++left > right) break;
        }
        return ans;
    }
};

```

搜索二维矩阵 II

题目：在行列递增矩阵中判断 target 是否存在。

```
/*
思路：从右上角开始搜索，当前值大于 target 就左移，小于 target 就下移。
关键数据结构：row 和 col 两个下标指针，用来在矩阵中逐步排除一整行或一整列。
注意：空矩阵要先处理；移动时保证 row、col 不越界。
*/

#include "MatrixTestUtils.hpp"

using namespace std;

class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int m = matrix.size();
        int row = 0, col = matrix[0].size() - 1;
        while(row < m && col >= 0){
            if(matrix[row][col] == target){
                return true;
            }else if(matrix[row][col] > target){
                col--;
            }else row++;
        }
        return false;
    }
};
```

旋转图像

题目：把 $n \times n$ 矩阵顺时针旋转 90 度。

```
/*
思路：先沿主对角线转置矩阵，再反转每一行，等价于顺时针旋转 90 度。
关键数据结构：二维 vector 原地交换元素，不需要额外矩阵。
注意：转置只遍历对角线一侧；反转每一行时不要漏掉所有行。
*/

#include "MatrixTestUtils.hpp"
#include <algorithm>

using namespace std;

class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int m = matrix.size();
        int n = matrix[0].size();
```

```

        for(int i = 0; i < m; i++){
            for(int j = i + 1; j < n; j++){
                swap(matrix[i][j], matrix[j][i]);
            }
        }
        for(int i = 0; i < m; i++){
            reverse(matrix[i].begin(), matrix[i].end());
        }
    }
};

```

No.7 链表

相交链表

题目：找到两个链表开始相交的节点。

```

/*
思路：双指针分别从 A、B 出发，走到末尾后切换到另一条链表，最终会在相交点或 nullptr 相遇。
关键数据结构：pA、pB 两个指针，用 A+B 和 B+A 的等长路径抵消长度差。
注意：判断相交看节点地址是否相同，不是看节点值是否相等。
*/

#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* getIntersectionNode(ListNode* headA, ListNode* headB) {
        ListNode *A = headA, *B = headB;
        // 如果没有相交的话，ab会同时为空
        while(A != B){
            A = !A? headB : A->next;
            B = !B? headA : B->next;
        }
        return A;
    }
};

void attachSharedTail(ListNode*& head, ListNode* shared) {
    if (head == nullptr) {
        head = shared;
        return;
    }
    tailOf(head)->next = shared;
}

```

反转链表

题目：反转单链表并返回新头节点。

```

/*
思路：用三个指针逐个改变 next 指向，把链表从前往后原地反转。
关键数据结构：ListNode 指针 prev、cur、next，用来保存前驱、当前节点和后继节点。
注意：改 cur->next 前必须先保存 next，否则后面的链表会断掉找不回来。
*/

#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode *pro = nullptr;
        ListNode *curr = head;
        while(curr){
            ListNode *temp = curr->next;

            curr->next = pro;

            pro = curr;
            curr = temp;
        }
        return pro;
    }
};

```

回文链表

题目：判断链表是否为回文结构。

```

/*
思路：快慢指针找中点，反转后半段，再从两端向中间逐个比较。
关键数据结构：slow/fast 定位中点，prev/cur 用来反转后半段链表。
注意：奇偶长度都要处理好；如果要求保持原链表，比较后应把后半段再反转回来。
找中点的判断：
    while(fast->next && fast->next->next)
此时右半边就是slow->next; (奇数不包括中间点)
*/

#include "LinkedListTestUtils.hpp"

ListNode *rv(ListNode *head){
    ListNode *pro = nullptr;
    ListNode *curr = head;
    while(curr){
        ListNode *temp = curr->next;

        curr->next = pro;

        pro = curr;
        curr = temp;
    }
}

```

```

    }
    return pro;
}

class Solution {
public:
    bool isPalindrome(ListNode* head) {
        ListNode *fast = head;
        ListNode *slow = head;

        while(fast->next && fast->next->next){
            fast = fast->next->next;
            slow = slow->next;
        }
        slow = rv(slow->next);
        fast = head;
        while(slow){
            if(slow->val != fast->val) return false;
            slow = slow->next;
            fast = fast->next;
        }

        return true;
    }
};

```

环形链表

题目：判断链表中是否存在环。

```

/*
思路：快慢指针同时走，slow 每次一步，fast 每次两步；有环时两者一定会相遇。
关键数据结构：slow 和 fast 两个 ListNode 指针，用来判断链表是否存在追及。
注意：循环条件要保证 fast 和 fast->next 非空，避免访问空指针。
    环形链表的两个题目用：while(fast && fast->next)
*/
#include "LinkedListTestUtils.hpp"

class Solution {
public:
    bool hasCycle(ListNode* head) {
        ListNode *fast = head;
        ListNode *slow = head;
        while(fast && fast->next){
            fast = fast->next->next;
            slow = slow->next;
            if(fast == slow) return true;
        }
        return false;
    }
};

```

```
    }
};
```

环形链表 II

题目：返回链表入环的第一个节点。

```
/*
思路：先用快慢指针判断是否有环；相遇后让一个指针回到头节点，两者同步走，重逢处就是入环点。
    slow: L + X
    fast: 2(L + X) = L + X + nC
    L = nC - X = (n-1)C + (C - X)
关键数据结构：slow、fast 两个指针用于找相遇点，再用 head 和 slow 同步定位入口。
注意：无环时直接返回 nullptr；第二阶段两个指针都一次走一步。
*/

#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* detectCycle(ListNode* head) {
        ListNode *fast = head;
        ListNode *slow = head;
        while(fast && fast->next){
            fast = fast->next->next;
            slow = slow->next;
            if(slow == fast) break;
        }
        if(fast == NULL || fast->next == NULL) return NULL;

        fast = head;
        while(fast != slow){
            fast = fast->next;
            slow = slow->next;
        }
        return fast;
    }
};
```

合并两个有序链表

题目：合并两个升序链表为一个升序链表。

```
/*
思路：双指针同时扫描两个有序链表，每次把较小节点接到结果链表尾部。
关键数据结构：dummy 虚拟头节点和 tail 尾指针，用来统一处理头节点和尾插。
注意：某一条链表先走完，剩下的节点可以直接整体接上。
*/
```

```
#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        ListNode dummy(0);
        ListNode *tail = &dummy;
        while(list1 && list2){
            if(list1->val < list2->val){
                tail->next = list1;
                tail = tail->next;
                list1 = list1->next;
            }else{
                tail->next = list2;
                tail = tail->next;
                list2 = list2->next;
            }
        }
        tail->next = list1? list1 : list2;
        return dummy.next;
    }
};
```

两数相加

题目：两个链表表示逆序整数，返回相加链表。

```
/*
思路：两个链表从头开始逐位相加，用 carry 保存进位，新建结果链表。
关键数据结构：dummy/tail 拼接结果节点，carry 记录当前位产生的进位。
注意：循环要覆盖 l1、l2 和最后 carry 三种情况，避免漏掉最终进位。
*/
```

```
#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode dummy(0);
        ListNode *tail = &dummy;
        int carry = 0;
        int sum = 0;
        while(l1 || l2){
            int a = l1? l1->val : 0;
            int b = l2? l2->val : 0;
            sum = a + b + carry;
            tail->next = new ListNode(sum % 10);
            carry = sum / 10;
            tail = tail->next;
            l1 = l1? l1->next : nullptr;
            l2 = l2? l2->next : nullptr;
        }
        if(carry > 0){
            tail->next = new ListNode(carry);
        }
        return dummy.next;
    }
};
```

```

    }
    if(carry) tail->next = new ListNode(carry);
    return dummy.next;
}
};

```

删除链表倒数第 N 个结点

题目：删除链表倒数第 n 个节点。

```

/*
思路：快慢指针保持 n 个节点间距，fast 到尾时 slow 正好在待删节点前面。
关键数据结构：dummy 虚拟头节点统一删除头节点情况，fast/slow 定位删除位置。
注意：先让 fast 走 n 步；删除后返回 dummy.next，不要直接返回原 head。
*/

#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode dummy(0, head);
        ListNode *fast = &dummy;
        ListNode *slow = &dummy;
        for(int i = 0; i <= n; i++){
            fast = fast->next;
        }
        while(fast){
            slow = slow->next;
            fast = fast->next;
        }
        ListNode *de = slow->next;
        slow->next = de->next;
        delete de;
        return dummy.next;
    }
};

```

两两交换链表中的节点

题目：每两个相邻节点交换一次。

```

/*
思路：用虚拟头节点，每次取出一对相邻节点，调整四条连接完成交换。
    交换完成之后要对pre进行更新
关键数据结构：dummy 和 pre 指针，用来定位每一组交换前的前驱节点。
注意：循环条件要保证 pre 后面至少还有两个节点；交换时先保存后续节点。
*/

```

```
#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode dummy(0, head);
        ListNode *pro = &dummy;
        while(pro->next && pro->next->next){
            ListNode *node1 = pro->next;
            ListNode *node2 = pro->next->next;

            node1->next = node2->next;
            node2->next = node1;
            pro->next = node2;

            pro = node1;
        }
        return dummy.next;
    }
};
```

K 个一组翻转链表

题目：每 k 个节点一组翻转链表。

```
/*
思路：每次先确认后面够 k 个节点，再把这一段局部反转并接回原链表。
关键数据结构：dummy/pre/end 指针定位每组边界，start/next 保存反转段和后续链表。
注意：不足 k 个节点时保持原顺序；反转前要先断开这一组的尾部。
*/

#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        (void)head;
        (void)k;
        // TODO: 在这里实现 K 个一组翻转链表算法。
        return nullptr;
    }
};
```

随机链表的复制

题目：复制带 random 指针的链表。

```
/*
思路：阶段 1：克隆节点并交织挂载
```

原链表: A -> B -> C

现链表: A -> A' -> B -> B' -> C -> C'

阶段 2: 处理克隆节点的 random 指针

如果原节点的 random 不为空,

克隆节点的 random 就是原节点 random 的 next

阶段 3: 物理拆分 (极其关键: 必须恢复原链表!)

恢复原链表

链接新链表

关键数据结构: unordered_map 保存新旧节点对应关系, 用来快速找到 random 指向的新节点。

注意: 复制链表不能复用原节点; random 为空时要保持 nullptr。

*/

```
#include "LinkedListTestUtils.hpp"
```

```
class Solution {
public:
    Node* copyRandomList(Node* head) {
        if(!head) return NULL;
        Node *p = head;
        while(p){
            Node *pp = new Node(p->val);
            pp->next = p->next;
            p->next = pp;
            p = pp->next;
        }
        p = head;
        while(p){
            Node *pp = p->next;
            if(p->random){
                pp->random = p->random->next;
            }
            p = pp->next;
        }
        p = head;
        Node *ans = p->next;
        Node *pp = ans;
        while(pp){
            p->next = pp->next;
            p = p->next;
            if(!p){
                return ans;
            }
            pp->next = p->next;
            pp = p->next;
        }
        return ans;
    }
};
```

排序链表

题目: 把链表按升序排序。

```
/*
思路：链表排序常用归并排序，先用快慢指针切成两半，递归排序后再合并。
关键数据结构：slow/fast 找中点，dummy/tail 合并两个有序子链表。
注意：切分时要断开前半段尾节点；递归出口是空链表或单节点。
*/
```

```
#include "LinkedListTestUtils.hpp"

ListNode* merge(ListNode* l1, ListNode* l2){
    ListNode dummy(0);
    ListNode *tail = &dummy;
    while(l1 && l2){
        if(l1->val < l2->val){
            tail->next = l1;
            l1 = l1->next;
            tail = tail->next;
        }else{
            tail->next = l2;
            l2 = l2->next;
            tail = tail->next;
        }
    }
    tail->next = l1? l1 : l2;
    return dummy.next;
}

class Solution {
public:
    ListNode* sortList(ListNode* head) {
        if(!head || !head->next) return head;
        ListNode *fast = head, *slow = head;
        while(fast->next && fast->next->next){
            fast = fast->next->next;
            slow = slow->next;
        }
        ListNode *mid = slow->next;
        slow->next = nullptr;
        ListNode *l = sortList(head);
        ListNode *r = sortList(mid);
        ListNode *ans = merge(l, r);
        return ans;
    }
};
```

合并 K 个升序链表

题目：合并多个升序链表为一个升序链表。

```
/*
思路：可以用小顶堆维护每条链表当前头节点，每次取出最小节点接到结果链表。
关键数据结构：priority_queue 小顶堆保存候选节点，用 dummy/tail 拼接答案。
```

注意：弹出节点后，如果它还有 next，要把 next 继续放入堆中。

```
*/

#include "LinkedListTestUtils.hpp"

class Solution {
public:
    ListNode* mergeKLists(std::vector<ListNode*>& lists) {
        (void)lists;
        // TODO: 在这里实现合并 k 个升序链表算法。
        return nullptr;
    }
};
```

LRU 缓存

题目：实现支持 O(1) get 和 put 的 LRU 缓存。

/*
思路：get/put 都把访问过的节点移动到链表头部，容量超限时删除链表尾部最久未使用节点。
三个重要的数据结构：

```
void addhead(Node *node)
void movehead(Node *node)
int removetail()
```

关键数据结构：unordered_map 负责 O(1) 找节点，双向链表负责 O(1) 调整新旧顺序。

注意：淘汰时要从 map 中删除被移除节点的 key，不是删除刚插入的 key。

```
*/

#include "LinkedListTestUtils.hpp"
#include <unordered_map>
using namespace std;

class LRUCache {
public:
    LRUCache(int capacity) {
        this->capacity = capacity;
        this->head = new Node(0, 0);
        this->tail = new Node(0, 0);
        head->next = tail;
        tail->pro = head;
        this->i = 0;
    }

    int get(int key) {
        auto it = mp.find(key);
        if(it == mp.end()){
            return -1;
        }
        movehead(it->second);
        return it->second->value;
    }
};
```

```
void put(int key, int value) {
    auto it = mp.find(key);
    if(it != mp.end()){
        it->second->value = value;
        movehead(it->second);
        return;
    }
    Node *temp = new Node(key, value);
    addhead(temp);
    mp[key] = temp;
    i++;
    if(i > capacity){
        i--;
        int temp = removetail();
        mp.erase(temp);
    }
}

private:
struct Node {
    int key, value;
    Node *next, *pro;
    Node(int key, int value): key(key), value(value){
        this->next = NULL;
        this->pro = NULL;
    }
};

int capacity;
int i;
Node *head;
Node *tail;
unordered_map<int, Node*> mp;

void addhead(Node *node){
    node->next = head->next;
    node->pro = head;

    head->next->pro = node;
    head->next = node;
}

void movehead(Node *node){
    node->pro->next = node->next;
    node->next->pro = node->pro;

    addhead(node);
}

int removetail(){
    Node *temp = tail->pro;
    int ans = temp->key;
    temp->pro->next = tail;
    tail->pro = temp->pro;
    delete temp;
    return ans;
}
```

```
    }  
};
```

No.8 二叉树

二叉树的中序遍历

题目：返回二叉树的中序遍历结果。

```
/*  
思路：迭代模拟递归中序遍历，先一路向左把当前节点入栈，再弹出访问，最后转向右子树。  
关键数据结构：stack 保存祖先节点，用来回到上一个还没访问的根节点。  
注意：入栈的是 curr 本身，不是 curr->left；循环条件要同时看 curr 和栈是否为空。  
中序遍历不需要循环之前就把root入栈  
*/  
  
#include "BinaryTreeTestUtils.hpp"  
#include<stack>  
#include<vector>  
  
using namespace std;  
  
class Solution {  
public:  
    vector<int> inorderTraversal(TreeNode* root) {  
        stack<TreeNode*> p;  
        TreeNode *curr = root;  
        vector<int> ans;  
        while(!p.empty() || curr){  
            while(curr){  
                p.push(curr);  
                curr = curr->left;  
            }  
            curr = p.top();  
            ans.push_back(curr->val);  
            p.pop();  
  
            curr = curr->right;  
        }  
        return ans;  
    }  
};
```

二叉树的最大深度

题目：返回二叉树的最大深度。

```
/*  
思路：后序递归，先分别求左右子树深度，当前节点深度就是 max(left, right) + 1。
```

关键数据结构：递归调用栈保存每层节点状态，用返回值把子树深度传回父节点。

注意：空节点深度为 0，这是递归出口；返回的是节点数，不是边数。

```
*/  
  
#include "BinaryTreeTestUtils.hpp"  
  
using namespace std;  
  
class Solution {  
public:  
    int maxDepth(TreeNode* root) {  
        if(!root) return 0;  
  
        int left = maxDepth(root->left);  
        int right = maxDepth(root->right);  
  
        return max(left, right) + 1;  
    }  
};
```

翻转二叉树

题目：交换每个节点的左右子树。

思路：前序递归（前序后序都可），当前节点先交换左右孩子，再递归翻转左右子树。
关键数据结构：递归调用栈负责遍历整棵树，swap 直接交换当前节点的左右指针。
注意：空节点直接返回；交换后 root->left 已经是原来的右子树。

```
*/  
  
#include "BinaryTreeTestUtils.hpp"  
  
using namespace std;  
  
class Solution {  
public:  
    TreeNode* invertTree(TreeNode* root) {  
        if(!root) return root;  
  
        swap(root->right, root->left);  
  
        invertTree(root->left);  
        invertTree(root->right);  
  
        return root;  
    }  
};
```

对称二叉树

题目：判断二叉树是否轴对称。

```

/*
思路：队列里成对保存需要镜像比较的节点，每次比较 left 和 right 是否同时为空、值是否相等。
关键数据结构：queue<pair<TreeNode*, TreeNode*>> 按层保存镜像节点对，用来同步比较左右两边。
注意：弹出队头后再判断；下一层要交叉入队：left->left 对 right->right, left->right 对 right->left。
*/

#include "BinaryTreeTestUtils.hpp"

using namespace std;

class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        if(!root) return true;
        queue<pair<TreeNode*, TreeNode*>> q;
        q.push({root->left, root->right});
        while(!q.empty()){
            auto [left, right] = q.front();
            q.pop();
            if(!left && !right) continue;
            if(!(left && right)) return false;
            if(left->val != right->val) return false;
            q.push({left->left, right->right});
            q.push({left->right, right->left});
        }
        return true;
    }
};

```

二叉树的直径

题目：返回二叉树任意两节点路径的最大边数。

```

/*
思路：后序递归求每个节点的左右深度，left + right 就是经过当前节点的最长路径边数。
关键数据结构：递归调用栈返回子树深度，maxheight 用来保存所有节点里最大的直径。
注意：递归返回深度给父节点，全局 maxheight 记录答案；直径按边数算，所以不用再 +1。
*/

#include "BinaryTreeTestUtils.hpp"

using namespace std;

int df(TreeNode* root, int *maxheight){
    if(!root) return 0;

```

```

    int left = df(root->left, maxheight);
    int right = df(root->right, maxheight);

    *maxheight = max(left + right, *maxheight);

    return max(left, right) + 1;
}

class Solution {
public:
    int diameterOfBinaryTree(TreeNode* root) {
        int maxheight = 0;
        df(root, &maxheight);
        return maxheight;
    }
};

```

二叉树的层序遍历

题目：按层从左到右遍历二叉树。

```

/*
思路：BFS 按层遍历，每轮先记录当前队列长度，只处理这一层的节点。
关键数据结构：queue 保存待访问节点，vector<int> temp 收集当前层结果。
注意：空树不要入队；一层处理完再把 temp 放进答案。
        层序遍历，在循环之前就要把root入队
*/

#include "BinaryTreeTestUtils.hpp"

using namespace std;

class Solution {
public:
    std::vector<std::vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> ans;
        queue<TreeNode*> q;

        if(root) q.push(root);

        while(!q.empty()){
            int q_size = q.size();
            vector<int> temp;
            for(int i = 0; i < q_size; i++){
                TreeNode *curr = q.front();
                q.pop();
                temp.push_back(curr->val);
                if(curr->left) q.push(curr->left);
                if(curr->right) q.push(curr->right);
            }

```

```
        ans.push_back(temp);
    }
    return ans;
}
};
```

将有序数组转换为二叉搜索树

题目：把升序数组转换成高度平衡 BST。

```
/*
思路：每次取有序数组中点作为根，左半边递归建左子树，右半边递归建右子树。
关键数据结构：vector 存有序数组，左右下标限定当前递归负责的区间。
注意：区间 left > right 时返回空；中点建树可以保证左右高度尽量接近。
*/

#include "BinaryTreeTestUtils.hpp"

using namespace std;

TreeNode* fm(vector<int>& nums, int left, int right){
    if(left > right) return nullptr;
    int mid = (left + right) >> 1;

    TreeNode* node = new TreeNode(nums[mid]);
    node->left = fm(nums, left, mid - 1);
    node->right = fm(nums, mid + 1, right);

    return node;
}

class Solution {
public:
    TreeNode* sortedArrayToBST(std::vector<int>& nums) {
        TreeNode *root = fm(nums, 0, nums.size() - 1);
        // TODO: 在这里实现将有序数组转换为二叉搜索树。
        return root;
    }
};
```

验证二叉搜索树

题目：判断二叉树是否满足 BST 性质。

```
/*
思路：递归给每个节点维护合法取值范围，左子树上界变成 root->val，右子树下界变成 root->val。
关键数据结构：递归调用栈向下传递 low/high 边界，用来约束整棵子树的取值范围。
注意：BST 要满足整棵子树范围，不只是和父节点比较；用 long long 防止边界值溢出。
```

```

*/

#include "BinaryTreeTestUtils.hpp"

using namespace std;

bool fm(TreeNode* root, long long low, long long height){
    if(!root) return true;

    if(root->val <= low || root->val >= height) return false;

    return fm(root->left, low, root->val) && fm(root->right, root->val, height);
}

class Solution {
public:
    bool isValidBST(TreeNode* root) {
        return fm(root, LONG_LONG_MIN, LONG_LONG_MAX);
    }
};

```

二叉搜索树中第 K 小的元素

题目：返回 BST 中第 k 小的节点值。

```

/*
思路：二叉搜索树中序遍历天然升序，迭代中序遍历时计数，第 k 次访问的节点就是答案。
关键数据结构：stack 模拟递归调用栈，用来保存一路向左经过但还没访问的节点。
注意：k 从 1 开始；每弹出一个节点才算真正访问一次。
*/

#include "BinaryTreeTestUtils.hpp"

using namespace std;

class Solution {
public:
    int kthSmallest(TreeNode* root, int k) {
        TreeNode *curr = root;
        stack<TreeNode*> q;
        int cnt = 0;
        while(!q.empty() || curr){
            while(curr){
                q.push(curr);
                curr = curr->left;
            }
            curr = q.top();
            q.pop();
            cnt++;
            if(cnt == k){
                return curr->val;
            }
        }
    }
};

```

```
        }
        curr = curr->right;
    }
    return -1;
}
};
```

二叉树的右视图

题目：返回从右侧能看到的节点值。

```
/*
思路：层序遍历，每层从左到右扫描，最后访问到的节点就是这一层的右视图。
关键数据结构：queue 保存当前层和下一层节点，用 q_size 固定每一层的边界。
注意：每层开始先固定 q_size；空树直接返回空数组，避免使用未初始化的 maxright。
*/
```

```
#include "BinaryTreeTestUtils.hpp"

using namespace std;

class Solution {
public:
    std::vector<int> rightSideView(TreeNode* root) {
        queue<TreeNode*> q;
        vector<int> ans;
        int maxright;
        TreeNode *curr;
        if(root) q.push(root);
        while(!q.empty()){
            int q_size = q.size();
            for(int i = 0; i < q_size; i++){
                curr = q.front();
                q.pop();
                maxright = curr->val;
                if(curr->left) q.push(curr->left);
                if(curr->right) q.push(curr->right);
            }
            ans.push_back(maxright);
        }
        return ans;
    }
};
```

二叉树展开为链表

题目：把二叉树原地展开成先序链表。

```
/*
思路：后序递归，先把左右子树各自展开，再把左链表接到 root->right，原右链表接到末尾。
关键数据结构：递归调用栈保存子树处理顺序，临时指针 temp 保存原右子树。
注意：拼接前要先保存原来的右子树；展开后所有 left 都要置空。
*/
```

```
#include "BinaryTreeTestUtils.hpp"

using namespace std;

class Solution {
public:
    void flatten(TreeNode* root) {
        if(!root) return;
        flatten(root->left);
        flatten(root->right);

        TreeNode *temp = root->right;
        root->right = root->left;
        root->left = nullptr;
        TreeNode *curr = root;
        while(curr->right) curr = curr->right;
        curr->right = temp;
    }
};
```

从前序与中序遍历序列构造二叉树

题目：根据前序和中序遍历还原二叉树。

```
/*
思路：前序第一个值是根；在中序中找到根的位置，用左子树节点个数切分左右数组并递归建树。
关键数据结构：vector 切分出左右子树的前序和中序序列，用来递归构造子树。
注意：vector 区间是左闭右开；index 表示左子树节点数量。
*/

#include "BinaryTreeTestUtils.hpp"

using namespace std;

class Solution {
public:
    TreeNode* buildTree(std::vector<int>& preorder, std::vector<int>& inorder) {
        if(preorder.empty()) return nullptr;
        TreeNode *root = new TreeNode(preorder[0]);
        int index = 0;
        for(int i = 0; i < preorder.size(); i++){
            if(preorder[0] == inorder[i]){
                index = i;
                break;
            }
        }
        root->left = buildTree(preorder.begin()+1, preorder.begin()+1+index, inorder.begin(), inorder.begin()+index);
        root->right = buildTree(preorder.begin()+1+index, preorder.end(), inorder.begin()+index+1, inorder.end());
        return root;
    }
};
```

```

    }
}

vector<int> leftpre(preorder.begin() + 1, preorder.begin() + index + 1);
vector<int> leftino(inorder.begin(), inorder.begin() + index);
root->left = buildTree(leftpre, leftino);
vector<int> rightpre(preorder.begin() + index + 1, preorder.end());
vector<int> rightino(inorder.begin() + index + 1, inorder.end());
root->right = buildTree(rightpre, rightino);

return root;
}
};

```

路径总和 III

题目：统计路径和等于 targetSum 的路径数量。

```

/*
思路：枚举每个节点作为路径起点，再向下递归累加路径和，遇到 targetSum 就计数。
关键数据结构：queue 枚举所有起点，递归调用栈沿着每个起点向下累加路径和。
注意：路径只能向下走；sum 用 long long，避免节点值累加溢出。
      因为 sum 使用的是值传递，因此不用回溯
*/

#include "BinaryTreeTestUtils.hpp"

using namespace std;

int df(TreeNode *node, long long sum, int targetSum){
    if(!node) return 0;
    sum += node->val;
    int count = 0;
    if(sum == targetSum){
        count++;
    }
    count += df(node->left, sum, targetSum);
    count += df(node->right, sum, targetSum);
    return count;
}

class Solution {
public:
    int pathSum(TreeNode* root, int targetSum) {
        if(!root) return 0;

        int ans = 0;
        queue<TreeNode*> q;
        q.push(root);
        while(!q.empty()){
            TreeNode *node = q.front();

```

```

        q.pop();
        ans += df(node, 0, targetSum);
        if(node->left) q.push(node->left);
        if(node->right) q.push(node->right);
    }
    return ans;
}
};

```

二叉树的最近公共祖先

题目：找到二叉树中两个节点的最近公共祖先。

```

/*
思路：DFS 回溯分别记录 root 到 p、q 的路径，再比较两条路径，最后一个相同节点就是最近公共祖先。
关键数据结构：vector 保存根到目标节点的路径，用来按下标逐个比较公共前缀。
注意：path/path_p/path_q 要引用传递；答案指针最好初始化为 nullptr。
        处理完一个左右节点之后，说明这个子节点已经结束了，要记得回溯一下。
*/

```

```

#include "BinaryTreeTestUtils.hpp"

```

```

using namespace std;

```

```

void df(TreeNode* root, TreeNode* p, TreeNode* q, vector<TreeNode*> &path,
vector<TreeNode*> &path_p, vector<TreeNode*> &path_q){
    if(!root) return;
    path.push_back(root);
    if(root == p){
        path_p = path;
    }
    if(root == q){
        path_q = path;
    }
    df(root->left, p, q, path, path_p, path_q);
    df(root->right, p, q, path, path_p, path_q);
    path.pop_back();
}

```

```

class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        vector<TreeNode*> path, path_p, path_q;
        TreeNode* ans;

        df(root, p, q, path, path_p, path_q);

        for(int i = 0; i < min(path_p.size(), path_q.size()); i++){
            if(path_p[i] == path_q[i]){
                ans = path_p[i];
            }
        }
        return ans;
    }
};

```

```
        }else{
            break;
        }
    }
    return ans;
}
};
```

二叉树中的最大路径和

题目：返回二叉树中任意路径的最大节点和。

```
/*
思路：后序递归，返回当前节点能向父节点贡献的最大单边路径，同时更新全局最大路径和。
关键数据结构：递归调用栈自底向上传回单边贡献值，全局变量记录遍历过程中最大的完整路径。
注意：返回给父节点只能选左右一边；全局答案可以同时取 left + root + right，负贡献要舍弃。
*/

#include "BinaryTreeTestUtils.hpp"

class Solution {
public:
    int maxPathSum(TreeNode* root) {
        (void)root;
        // TODO: 在这里实现二叉树中的最大路径和。
        return 0;
    }
};
```